第 12 章 项目案例：一个会看会听的机械臂

> 本章目标：把一个真实可运行的项目拆开，看模型、Agent、控制程序和人的判断如何在同一条链路里各司其职；同时学会读一份“诚实的工程状态表”。

前面十一章讲的都是通用方法。这一章换一个角度：从一台真实的 RoArm-M2 机械臂出发，看课程里的概念分别落在哪一层。本项目本身只是一个教学模块的实验载体，不是课程主体；但它足够完整，可以当作“从想法到可用系统”的样本。

12.1 项目是什么

RoArm-Playground 把一个网页工作台、一台 RK3588 单板计算机和一条四轴机械臂连在一起：

- 浏览器里能看到摄像头画面、机械臂真实关节角度和一个三维姿态模型；
- 18 项互动实验共用同一套“感知—判断—动作”接口；
- 部分感知在 RK3588 本地完成（人脸、颜色、标记、物品、跟踪、拼接、找不同），部分在上位机完成（手势、可选的本地语言模型）；
- 一条断网也能用的局域网链路，课堂现场不需要公网。

需要先说明边界：这是一个个人内部使用的小项目，不是工业产品。机械臂存在可见到位误差，三维模型是运动几何示意而非标定结果，部分实验只完成到“软件与样本验证”层级。这些限制不是缺陷包装，而是后面 12.6 节要讲的教学材料。

12.2 硬件与链路

部件	角色	关键事实
RoArm-M2-S 机械臂	执行机构	四轴（底座/肩/肘/夹爪），ESP32 控制板，出厂固件通过 USB 串口 115200/8N1 通信
RK3588 单板机	板端大脑	Ubuntu 20.04 / Python 3.8 / OpenCV 4.8，负责视频、轻量视觉、中文语音与串口运动
USB 摄像头	眼睛	固定在夹爪附近，随末端一起运动；默认 640×480 MJPEG

部件	角色	关键事实
USB 麦克风	耳朵	采集中文口令与拍手节奏
演示笔记本	上位机	运行手势模型与可选本地语言模型，通过 Wi-Fi 与 RK 通信

链路是一条单向的“请求—执行—反馈”回路：

```
浏览器 / 上位机
      │ Wi-Fi (HTTP / JSON)
      ▼
  RK3588 服务
      │ USB 串口 (T101/T102/T105...)
      ▼
 机械臂 ESP32 ──► 四个舵机
      ▲                │
      └── 编码器反馈 ◄──┘
```

> 要点：网页发出的“目标角度”和串口读回的“实测角度”是两件事。项目里状态接口刻意把它们分开，模型动画只跟随实测反馈，不拿目标冒充结果。

为什么不让 ESP32 直接开 Wi-Fi、浏览器直连机械臂？这是本项目最早的一个设计决定。早期路线走 ESP32 无线页，后来改成 **RK3588 串口**：单板机可以同时处理视频与视觉，避免把摄像头数据先传回 ESP32；串口协议是厂家固件原生支持的，少一层自定义无线协议；出问题时分得清是网络、服务还是串口。这个取舍很好讲给学生：**“能连上”不等于“应该这样连”**。

12.3 软件结构

```
rk3588/                板端：网页、视频、视觉、串口控制
  playground.py        模式协调、动作程序、事件与状态接口
  camera_server.py     MJPEG 视频与快照
  vision.py            视觉引擎（人脸/颜色/标记/物品/跟踪/拼接/找不同）
  follow_control.py    连续跟随的滤波与控制曲线
  arm_serial.py        厂家串口协议适配
  control.html         工作台页面
  playground.js        前端交互与三维模型
host/                  上位机：手势、本地语言模型
  worker.py            手势/语音/拍手 worker
  director.py          自然语言 → 受限动作计划的预览服务
  manager.py           8082 模型管理与进程监督
scripts/               模型下载与环境准备
```

```
tests/                自动化测试
docs/                 功能、部署、实测状态与逐项验收手册
```

这个分层值得学生注意两点。

第一，**“网页”不是装饰**。`playground.js` 里有请求合并：拖动滑条时浏览器约每 16 毫秒合并一次目标，持续拖动约 33 毫秒发一次，只保留一个在途请求和最新待发值。不合并会发生什么？拖动结束后机械臂还在一格一格执行过时的中间目标。这是典型的“界面问题其实是控制问题”。

第二，**感知与控制解耦**。视觉线程只处理最新一帧，运动线程按 25 毫秒调度，两者不排队。视觉慢于控制时跳过旧帧，而不是补发。这个设计让“检测到目标”和“机械臂怎么动”可以分别调参、分别验收。

12.4 18 项实验的共同骨架

18 项实验看起来五花八门，实际上几乎是同一个循环的变体：

```
采集一帧 → 提取信息（框、类别、编号、文字、音频特征）
        → 做判断（目标在哪、置信度够不够、命令是什么）
        → 产生目标（关节角增量 / 灯光 / 程序步骤）
        → 受限执行（限位、限速、可停止）
        → 读回实测 → 下一轮
```

类别	实验	感知方式	输出的动作
视觉	找人致意、颜色跟随、标记寻宝、物品识别、框选追踪	YuNet / HSV 分割 / AprilTag / YOLOX-Nano / 光流+模板	双轴朝向或仅标注
听觉	中文口令、拍手节奏	Vosk 中文小模型 / 音量上升检测	预置动作或灯光节奏
手势	手势遥控、猜拳	MediaPipe Gesture Recognizer（上位机）	映射为动作
编排	示教回放、图形化积木	实测姿态记录	串行/循环/条件动作程序
影像	全景拼接、找不同	特征拼接 / 帧差	只产出图像结果
生成	自然语言导演、Agent 改规则	本地 Qwen 小模型 / 编程 Agent	受限动作计划的预览与执行
开发中	黑泡沫抓取、IMU 头部随动	暗色轮廓 / 模拟输入	尚未完成实物闭环

三个例子能把“感知—判断—动作”的差异讲清楚：

- **颜色跟随**是阈值分割，几行 OpenCV 就能跑，速度最快，但换光照就失效；
- **人脸跟随**用的是神经网络检测器（YuNet），比早先的 Haar 级联更稳，但只识别人脸位置，不认人；
- **自然语言导演**把一个 1.5B 参数的小语言模型放在上位机，把中文句子转成最多 12 步的动作计划，真正执行前必须人工预览。

同一条链路上，越往后“模型”占比越高，“规则与约束”也必须越重。这是本章最重要的一句话。

12.5 语言模型如何变成安全动作

“让机械臂听懂人话”听起来是大模型的任务，但工程上真正难的是**不让它乱来**。项目里的做法是三层收窄：

第一层：只开放有限动作。 模型只能输出四类步骤——灯光亮度（0..255 整数）、关节方向增量（左右/上下/开合，0..20 度）、等待（0..5 秒）、致意。没有“任意移动”“抓取”“旋转”。

第二层：模型讲语义，程序定符号。 早期让模型直接输出正负角度，结果左转右转、夹爪开合的符号经常反。后来改成模型只填 `direction: left/right/up/down/open/close`，控制器用一张固定的方向绑定表换算成物理符号。模型不再有机会算错正负。

第三层：独立校验与人工预览。 服务返回的 JSON 会被逐字段检查：动作类型、角度上下限、等待范围、字段是否多余；最多 12 步；超能力请求返回空计划并说明。网页只展示预览，用户点击后才执行。

最终计划长这样（示意）：

```
{
  "steps": [
    {"type": "led", "value": 255},
    {"type": "joint", "joint": "base", "delta": 10},
    {"type": "wait", "seconds": 1},
    {"type": "greet"}
  ],
  "explanation": "亮灯，向左转十度，等待后致意"
}
```

这段 JSON 里没有一处需要模型“算对”：方向由程序解释，范围由校验保证，执行由人确认。**把确定性从模型手里拿走，交给代码和人工**，这是所有高风险场景（机械、金融、发文、部署）通用的降风险手法。

> 要点：小模型能做的是“把话翻译成结构化意图”，不是“负责”。负责的永远是校验代码与按下执行键的人。

12.6 诚实的工程：状态表怎么读

项目里有一份逐项状态表（`docs/playground/status.md`），把每项功能拆成“已完成的软件/样本测试”和“尚需现场确认”。这个拆法本身就是教学内容——它区分了四种完全不同的“完成了”：

层级	含义	例子
说得出	代码写好了、文档写了	“导演服务支持 12 步计划”
跑得通	在本机或离线样本上真实运行过	官方图片推理出 dog/bicycle/car
测得过	自动化测试或固定回归通过	87 项→96 项测试全通过
现场看得见	真人、真物、真环境闭环	用户接回舵机后现场确认网页控制正常

课程要求学生交付时按这四层报告，而不是笼统写“已完成”。这不是形式主义：自动测试用的是假串口、公开样本和注入的 API，它们证明不了真人挥手能不能被识别。

项目里保留了两个很有价值的失败样本：

1. 早期用 Haar 级联做人脸检测，在**没有人的墙面和桌椅**上误检了两张“脸”。换成 YuNet 后，同一张画面为零检测。失败没有被删除，而是写进了文档，提醒后续测试必须先验证“空地没有目标”。
2. 语言模型最初直接输出正负角度，多次把左转写成右转、把夹爪张开写成合拢。修法不是换更大的模型，而是改协议，让模型不碰符号。失败记录同样保留。

> 要点：一个项目是否可信，不取决于它有多少成功记录，而取决于它如何记录失败。

12.7 现场改规则：Agent 在这里做什么

第 16 项实验叫“现场 Agent 改规则”：给出一个可快速撤回的单点需求，例如“把默认跟随颜色改成绿色”或“让某个口令点亮更暗的灯”，然后让编程 Agent 完成“读代码 → 改参数 → 跑测试 → 部署 → 现场复现效果”的完整闭环。

这一步才是 VibeCoding 在这个项目里的真实位置：**Agent 不是操作机械臂，而是修改控制机械臂的规则**。学生能观察到的证据有三样：

- 一份可审查的代码 diff；
- 相关测试的实际输出（颜色改 `vision.py`、语音改 `host/worker.py`、控制改动跑全量测试）；

- 用相同输入再次演示时的行为变化。

页面上的参数开关不能冒充“Agent 写了代码”。这条验收标准可以直接搬给学生当作业规范。

12.8 学生可以带走什么

把本章压缩成四句话：

1. **分层是省事的来源**：感知、判断、动作、界面各自可替换，出问题能定位。
2. **越智能的环节，越需要外部的约束与校验**，而不是指望模型自觉。
3. **“完成”有层级**：说得出、跑得通、测得过、现场看得见，报告时要写清在哪一层。
4. **失败要留下证据**，它是下一次测试的起点，也是可信度的一部分。

课后题

1. 本项目为什么放弃“ESP32 开热点、浏览器直连”的方案，改用 RK3588 串口中转？列出至少两个理由，并说出它带来的一个新代价。
2. 找一份你自己写过的代码或项目文档，按“说得出 / 跑得通 / 测得过 / 现场看得见”四层重新标注它的完成状态。
3. 如果让你给语言导演增加一个“夹爪开合”动作，需要同时改动哪几处（动作 schema、校验、方向绑定、前端预览）？任选两处说明你会怎么改。
4. 设计一个“现场改规则”的题目，要求它：单点、可撤回、效果肉眼可见、有对应自动化测试。
5. 思考题：机械臂项目里哪些判断适合交给模型，哪些必须留给代码和人工？给出三条划分原则。